

Abstract — Investment research requires gathering real financial data, interpreting it through multiple analytical lenses, and synthesising the findings into an actionable recommendation. These tasks cannot be compressed into a single LLM prompt: the model has no access to current financial figures, conflating descriptive analysis with prescriptive thesis construction causes selective evidence weighting, and a model anchors on its own output when asked to critique it. This paper presents a six-step multi-step LLM agent that chains a real-data tool call with five sequential LLM reasoning steps — each consuming the structured JSON output of its predecessor — to produce a professional equity research brief. Evaluated on Apple Inc. (AAPL), the agent demonstrates that step separation, adversarial role assignment, and explicit assumption surfacing each materially improve the specificity and reliability of the output relative to a single-prompt baseline. **Keywords** — large language models, multi-step reasoning, tool use, financial NLP, prompt chaining, equity research

I. INTRODUCTION

Large language models exhibit strong language understanding and general reasoning capabilities, but applying them to real-world analytical tasks reveals two structural gaps. First, LLMs have no access to current external data: asking a model to evaluate a stock's valuation without providing real financial figures produces hallucinated numbers that appear plausible but are factually wrong [4]. Second, complex analytical tasks decompose into cognitively distinct subtasks that produce better results when handled separately. Wei et al. [2] demonstrate that prompting a model to reason through intermediate steps before answering substantially improves accuracy on complex tasks; this insight motivates the chaining approach taken here.

Investment research is a natural fit for multi-step chaining. The task requires: (1) resolving a natural language query into a machine-readable form suitable for a tool call; (2) fetching real financial data the LLM cannot supply; (3) interpreting that data through the lens of fundamental analysis; (4) constructing a forward-looking investment thesis; (5) stress-testing the thesis from an adversarial perspective; and (6) synthesising the outputs into a structured report. Each step depends on the previous one, and no step can be skipped without breaking downstream reasoning. This paper describes the design, implementation, and evaluation of such an agent, and reflects on the architectural insights gained during development.

II. RELATED WORK

The ReAct framework [1] introduced interleaving reasoning traces with tool calls, establishing the pattern that directly inspires this agent's architecture: a tool call (Step 2) provides grounded observations that feed a reasoning chain (Steps 3–5). Toolformer [3] demonstrated that tool calls should be structurally separate from LLM reasoning steps, which motivates the clean boundary between Step 2 and Step 3 in our design. In the financial domain, BloombergGPT [4] established that finance tasks require grounded data inputs rather than relying on LLM training knowledge, and Lopez-Lira and Tang [5] showed that LLM-based analysis of financial text produces predictive signal above chance. FinGPT [6] informed the use of structured JSON output for financial reasoning steps.

III. SYSTEM ARCHITECTURE

A. Chain Overview

The agent maintains a single Python dictionary as shared state. Each step reads from the state, performs its task, and writes its output back, so all downstream steps accumulate context. Table I summarises the six-step chain.

TABLE I: Agent Chain Structure

Step	Type	Input (from state)	Output (to state)
1	LLM	<code>user_query</code>	<code>parsed_query</code>
2	Tool	<code>parsed_query.ticker</code>	<code>financial_data</code> (~25 metrics + news)
3	LLM	<code>financial_data</code> , <code>parsed_query</code>	<code>fundamental_analysis</code>
4	LLM	<code>fundamental_analysis</code> , <code>financial_data</code>	<code>investment_thesis</code>
5	LLM	<code>investment_thesis</code> , <code>fundamental_analysis</code> , <code>financial_data</code>	<code>thesis_critique</code>
6	LLM	all state	<code>final_report</code> (Markdown string)

B. Step Design

Step 1 — Query parsing. The user's natural language input (e.g., "Is Apple a good long-term investment?") is passed to an LLM with a strict JSON-only system prompt that enforces a fixed output schema: `{company_name, ticker, analysis_focus, key_questions, time_horizon}`. The temperature is set to 0.2 for deterministic extraction. This step exists because the entire chain — including the tool call — depends on a reliable ticker symbol. Without structured extraction, fuzzy input cannot drive Step 2.

Step 2 — Financial data fetch (tool call). Using the ticker from Step 1, the agent calls Yahoo Finance via the `yfinance` Python library and retrieves approximately 25 financial metrics: current price, P/E and forward P/E, PEG ratio, revenue and earnings growth, gross/operating/net margins, return on equity, debt-to-equity, current ratio, analyst consensus target, beta, dividend yield, 52-week range, and five recent news headlines. This is a tool call, not an LLM call, because language models cannot produce accurate

current financial figures. Error handling catches invalid tickers, sets a `tool_call_success` flag, and degrades gracefully rather than crashing the chain.

Step 3 — Fundamental analysis. The LLM receives the financial metrics from Step 2 and the user's focus and questions from Step 1. Its system prompt assigns the role of senior equity analyst and enforces a JSON schema with fields for `valuation_assessment` (constrained to three values), `financial_strengths`, `financial_weaknesses`, `key_metrics_summary`, `competitive_position`, `red_flags`, and a `data_quality_note`. This step is separate from Step 4 because descriptive analysis and prescriptive thesis construction are cognitively distinct — combining them in testing caused the model to adopt a stance early and selectively weight evidence to support it.

Step 4 — Investment thesis. The LLM receives the fundamental analysis from Step 3, price context from Step 2, and time horizon from Step 1. It outputs a bull case, a bear case, a `recommended_action`, and critically a `key_assumptions` list of 3–5 items. The assumptions list is the structural dependency that connects Step 4 to Step 5: without it, the critique step has no specific claims to challenge and defaults to generic risk commentary. This design decision emerged from testing and is discussed further in Section VI.

Step 5 — Stress test. The LLM is assigned the role of risk manager and instructed to challenge every assumption from Step 4 by name. It also receives the `red_flags` from Step 3 and the news headlines from Step 2 to check whether the thesis ignored recent corporate events. Temperature is raised to 0.5 to encourage diverse challenges. The output includes `challenged_assumptions` with severity ratings, `scenario_analysis` (best/base/worst case), a `final_risk_rating`, and a `thesis_validity_score` (1–10).

Step 6 — Report synthesis. The LLM receives all prior outputs and synthesises them into a structured Markdown research brief with fixed section headers specified verbatim in the system prompt. The recommendation and risk rating are explicitly injected at the top of the user prompt rather than buried in nested JSON, ensuring they appear prominently in the output. Temperature is 0.6 to allow natural prose variation.

IV. TOOL INTEGRATION

The tool used is **yfinance**, a Python library that queries Yahoo Finance's public endpoint without requiring an API key. It was chosen over Alpha Vantage and Polygon.io because it is fully free-tier, returns data as a Python dict that integrates directly with the shared state object, and covers all metrics required for equity fundamental analysis. Its output enters the chain at Step 2 and is consumed by Steps 3, 4, 5, and 6. The data flows through the chain unmodified — each LLM step selects the subset of fields relevant to its task — which maintains a single authoritative source of financial truth throughout the reasoning chain.

V. EXPERIMENTAL RESULTS

The agent was evaluated on the query *"Analyse Apple stock for long-term growth investment"*. Step 1 correctly extracted ticker `AAPL`, focus `growth`, and horizon `long-term (3+`

`years)`. Step 2 fetched a current price of `\$293.32`, P/E of `35.47`, market capitalisation of `\$4.31T`, revenue growth of `16.6% YoY`, and net profit margins of `27.2%`. Step 3 classified the stock as **overvalued**, identifying four financial strengths (notably a `47.9% gross margin` and `141% ROE`) and five red flags including a `79.5x debt-to-equity ratio` and a current ratio of `1.07`. Step 4 produced a **neutral/accumulate** thesis with five named assumptions, including services revenue growth offsetting iPhone saturation. Step 5 challenged all five assumptions with specific severity ratings and produced a **HIGH** risk rating with validity score of **4/10**, noting that Vision Pro adoption data directly contradicted the innovation optionality assumption. Step 6 synthesised a 16,932-character research brief complete with sensitivity analysis. End-to-end latency was approximately 35 seconds.

VI. LIMITATIONS

The most significant limitation is silent cascade degradation. When Step 2 returns incomplete data — common for smaller-cap and international tickers with limited Yahoo Finance coverage — downstream steps produce confident-sounding outputs that are factually wrong. The chain has no circuit-breaker that pauses when data quality falls below a minimum threshold. A solution would be a Python assertion layer between steps that validates LLM-stated figures against the raw Step 2 dict before passing conclusions downstream.

A second limitation is that Step 5's critique is performed by the same model family that constructed Step 4's thesis. Although the adversarial system prompt reduces anchoring, it cannot eliminate it. In practice, Step 5 challenged peripheral assumptions while leaving the overall stance intact. A genuinely independent critique would require either a different model or a human reviewer.

Third, the agent lacks a peer benchmarking step. Assertions such as "margins are high" or "P/E is elevated" are qualitative and unbenchmarked. Inserting a peer fetch step between Steps 2 and 3 — retrieving the same metrics for three to five sector competitors — would transform these into quantitative claims with sector context.

VII. CONCLUSION

This paper presented a six-step investment research agent demonstrating that multi-step LLM chaining genuinely outperforms single-prompt approaches for tasks requiring external data, multi-perspective analysis, and self-critique. The most non-obvious architectural insight was that requiring Step 4 to explicitly surface its own assumptions as structured output — rather than embedding them implicitly in prose — transformed Step 5 from a generic risk commentary generator into a specific, named-assumption challenger. This principle generalises: any step that constructs an argument in a multi-step chain should be required to surface its assumptions as structured output so the next step can challenge them by name. A second insight is that the `tool_call_success` flag and graceful degradation pattern are necessary but insufficient — the chain needs active data quality checks, not just passive fallbacks, to prevent cascading errors from propagating invisibly through downstream reasoning steps.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," *ICLR 2023*.
- [2] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," *NeurIPS 2022*.
- [3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *NeurIPS 2023*.
- [4] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann, "BloombergGPT: A large language model for finance," *arXiv:2303.17564*, 2023.
- [5] A. Lopez-Lira and Y. Tang, "Can ChatGPT forecast stock price movements? Return predictability from a large language model," *arXiv:2304.07619*, 2023.
- [6] H. Yang, X. Liu, and C. D. Wang, "FinGPT: Open-source financial large language models," *arXiv:2306.06031*, 2023.
- [7] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *NeurIPS 2022*.

Appendix: Prompt Design Explanation

This appendix documents the full system prompt and user prompt for each LLM step in the chain, explains why each prompt is shaped the way it is, and shows one example of a prompt that was changed after testing.

STEP 1 — PARSE USER QUERY

System prompt

You are a financial query parser. Extract structured information from the user's investment research request.

Respond with valid JSON only — no markdown, no explanation, no prose. Use exactly this schema:

```
{
  "company_name": "<full company name>",
  "ticker": "<stock ticker symbol, e.g. AAPL>",
  "analysis_focus": "<one of: growth | value | dividend | risk | general>",
  "key_questions": ["<question 1>", "<question 2>", "<question 3>"],
  "time_horizon": "<e.g. short-term (<1 year) | medium-term (1-3 years) | long-term (3+ years)>"
}
```

Rules:

- If the user does not supply a ticker, infer it from the company name.
- If the user does not specify a time horizon, default to "medium-term (1-3 years)".
- key_questions must have 3 to 5 items that reflect what the user actually wants to know.
- analysis_focus must be exactly one of the five options listed above.

User prompt (template)

Parse this investment research request and return JSON:

"<user's raw query>"

Design rationale

The instruction "Respond with valid JSON only — no markdown, no explanation, no prose" is the most critical constraint. Without it, the model typically wraps the JSON in a sentence like "Here is the parsed result:" which breaks downstream JSON parsing. The explicit schema with field names and example values prevents the model from inventing its own field names. The `analysis_focus` constraint to five exact values ensures Step 3's prompt can reference it without needing to normalise free-form text. Temperature is set to 0.2 (lowest in the chain) because this step requires deterministic extraction, not creative reasoning. The output of this step directly drives the tool call in Step 2 — an incorrect ticker here breaks the entire chain.

STEP 3 — FUNDAMENTAL ANALYSIS

System prompt

You are a senior equity analyst performing fundamental analysis on real financial data.

Respond with valid JSON only — no markdown, no prose. Use exactly this schema:

```
{
  "valuation_assessment": "<undervalued | fairly_valued | overvalued>",
  "valuation_reasoning": "<2-3 sentence explanation citing specific metrics>",
  "financial_strengths": ["<strength 1>", "<strength 2>", "..."],
  "financial_weaknesses": ["<weakness 1>", "<weakness 2>", "..."],
  "key_metrics_summary": {
    "profitability": "<concise assessment>",
    "growth": "<concise assessment>",
    "liquidity": "<concise assessment>",
    "leverage": "<concise assessment>"
  },
  "competitive_position": "<2-sentence assessment of competitive moat and sector standing>",
  "red_flags": ["<flag 1>", "..."],
  "data_quality_note": "<note any missing or suspicious data fields and their potential impact>"
}
```

Rules:

- Base every claim on the numbers provided. Do not hallucinate metrics.
- If a metric is missing (null / N/A), say so in data_quality_note rather than estimating.
- financial_strengths and financial_weaknesses must each have 2-5 items.

User prompt (template)

Perform fundamental analysis for <company_name> (<ticker>).

Sector: <sector> | Industry: <industry>

Analysis focus: <analysis_focus>

User's specific questions: <key_questions>

Note: Real financial data WAS/WAS NOT successfully retrieved via yfinance.

Key financial metrics:

<metrics dict — subset of ~20 fields>

Business summary (first 600 chars):

<business_summary>

Provide your structured fundamental analysis as JSON.

Design rationale

The `data_quality_note` field was added after testing revealed a failure mode: when several metrics were null (common for small-cap stocks), the model silently invented plausible-sounding figures rather than acknowledging the gap. Adding an explicit field for data quality issues forces the

model to flag uncertainty rather than paper over it. The user's `key_questions` from Step 1 are injected here so the analysis is targeted to what the user actually asked rather than generic. The `red_flags` field is structurally important — Step 5 reads it directly to seed the critique.

STEP 4 — INVESTMENT THESIS

System prompt

You are a portfolio manager constructing an investment thesis.
Given a fundamental analysis and supporting financial data, build a balanced bull and bear case.

Respond with valid JSON only — no markdown, no prose. Use exactly this schema:

```
{
  "overall_stance": "<bullish | bearish | neutral>",
  "conviction_level": "<high | medium | low>",
  "investment_summary": "<1-2 sentence plain-English summary of your position>",
  "bull_case": {
    "narrative": "<2-3 sentence argument for why the stock could outperform>",
    "catalysts": ["<catalyst 1>", "<catalyst 2>", "..."],
    "upside_target_rationale": "<brief explanation of what would drive price higher>"
  },
  "bear_case": {
    "narrative": "<2-3 sentence argument for why the stock could underperform>",
    "risks": ["<risk 1>", "<risk 2>", "..."],
    "downside_scenario": "<brief description of a realistic negative outcome>"
  },
  "key_assumptions": ["<assumption the thesis rests on 1>", "..."],
  "key_monitoring_metrics": ["<what to watch 1>", "..."],
  "recommended_action": "<buy | accumulate | hold | reduce | avoid>",
  "position_sizing_suggestion": "<small | medium | large | none>"
}
```

Rules:

- `bull_case.catalysts` and `bear_case.risks` must each have 2-4 items.
- `key_assumptions` must have 3-5 items — these will be stress-tested in the next step.
- Do not recommend "buy" with low conviction; use "accumulate" instead.
- Position sizing must match conviction: high-medium/large, low-small/none.

User prompt (template)

Build an investment thesis for `<company_name>` (`<ticker>`).

Context:

Sector : `<sector>`
Analysis focus: `<analysis_focus>`
Time horizon : `<time_horizon>`

Price & analyst data:

```
<price context dict>

Business overview (truncated):
<business_summary>

Fundamental analysis (from Step 3):
<fundamental_analysis dict>

Construct a balanced bull and bear investment thesis as JSON.
```

Design rationale — includes prompt iteration example

Version 1 (original): The original Step 4 system prompt did not require an explicit `key_assumptions` field. It only asked for bull/bear cases and a recommendation. The output was structurally fine but Step 5 had nothing concrete to challenge — the critique became generic ("consider macroeconomic conditions") rather than specific ("the thesis assumes services revenue will offset iPhone saturation — this rests on the unverified claim that Apple TV+ subscriber growth is accelerating").

What it produced: Step 5 outputs like: *"The thesis does not adequately consider macro risks."* — too vague to be useful.

What was changed: Added `key_assumptions` as a required field with 3-5 items in the schema, and added the rule note *"these will be stress-tested in the next step"* to signal to the model that this field has downstream consequences.

Why the new version works better: Step 5 now receives the exact list of assumptions from Step 4's JSON and is instructed to challenge all of them by name. This creates a concrete data dependency between the two steps and produces specific, named critiques rather than generic risk warnings.

STEP 5 — CRITIQUE AND STRESS TEST

System prompt

You are a risk manager stress-testing an investment thesis before it goes to committee. Your job is to challenge every assumption, find logical gaps, and identify what could go wrong.
Be specific and adversarial — a "great job" review is worthless.

Respond with valid JSON only — no markdown, no prose. Use exactly this schema:

```
{
  "thesis_validity_score": <integer 1-10>,
  "strongest_points": ["<point 1>", "..."],
  "challenged_assumptions": [
    {
      "assumption": "<exact assumption from the thesis>",
      "challenge": "<why this assumption might be wrong>",
      "severity": "<high | medium | low>"
    }
  ],
  "missing_analysis": ["<what was not considered 1>", "..."],
  "macro_risks": ["<macro or sector-level risk 1>", "..."],
  "scenario_analysis": {
    "best_case": "<brief description and what
```

```

drives it>",
  "base_case": "<brief description and what
drives it>",
  "worst_case": "<brief description and what
drives it>"
},
"final_risk_rating": "<low | medium | high |
very_high>",
"recommendation_to_analyst": "<what to in-
vestigate or re-examine before acting on this
thesis>"
}

Rules:
- challenged_assumptions must cover ALL as-
sumptions from the thesis, not a subset.
- thesis_validity_score of 8+ requires strong
evidence from the data; 5- requires specific
critique.
- Do not simply restate what the thesis said.
Challenge it.

```

User prompt (template)

```

Stress-test this investment thesis for <com-
pany_name> (<ticker>).

Time horizon under review: <time_horizon>

Investment thesis to challenge (from Step 4):
<investment_thesis dict>

Supporting financial metrics (from Step 2):
<risk_metrics dict - selected subset>

Red flags already identified in fundamental
analysis (from Step 3):
<red_flags list>

Recent news headlines (from Step 2 tool call):
<news_headlines list>

Challenge every key_assumption listed in the
thesis. Produce a rigorous risk assessment as
JSON.

```

Design rationale

The adversarial role ("risk manager", "Be specific and adversarial — a great job review is worthless") is essential. Without it, the model defaults to a balanced tone that validates rather than challenges. Injecting the news headlines from Step 2 into this step's context checks whether the thesis accounted for recent events — the connection between the real-world tool call and the critique step. Temperature is set to 0.5 (slightly higher than other steps) to encourage more diverse challenges rather than repeating the same risk in different words.

STEP 6 — FINAL REPORT

System prompt

```

You are a senior investment research writer.
Synthesise all provided analysis into

```

```

a professional equity research brief that a
portfolio manager can act on directly.

```

Write in polished financial prose. Use the section headers below exactly as written. Cite specific numbers where available. Be direct about the recommendation.

Required sections (use these exact headers in this order):

```

## Executive Summary
## Company Overview
## Financial Analysis
## Investment Thesis
## Risk Assessment
## Key Metrics to Monitor
## Conclusion & Recommendation

```

Do not include a disclaimer beyond one line at the very end.

User prompt (template)

```

Write a full equity research brief for <com-
pany_name> (<ticker>).

KEY SIGNAL FIGURES (cite these, do not invent
alternatives):
<headline_figures dict>

USER'S ORIGINAL QUESTIONS:
<key_questions>

RECOMMENDATION (must appear in Executive Sum-
mary and Conclusion):
Action: <recommended_action>
Stance: <overall_stance>
Conviction: <conviction_level>
Risk Rating: <final_risk_rating>
Validity: <thesis_validity_score>/10

FUNDAMENTAL ANALYSIS (Step 3): <full dict>
INVESTMENT THESIS (Step 4): <full dict>
RISK CRITIQUE (Step 5): <full dict>

Sector & Industry, business overview, recent
news...

Now write the full research brief with all
required sections.

```

Design rationale

The required section headers are specified verbatim in the system prompt to ensure consistent structure across different queries and stocks. Without this, early test runs produced arbitrarily ordered sections — some omitting risk entirely, others burying the recommendation in the final paragraph. The recommendation, stance, and risk rating are explicitly injected at the top of the user prompt (not just present in the nested JSON dicts) so the model treats them as primary directives rather than buried context. Temperature is 0.6, the highest in the chain, because the final report benefits from natural prose variety rather than formulaic repetition.